TECHNICAL PROJECT REPORT

Colony: One

Building and Evaluating a Compact Neural Network

A practical account of model development, data generation, training experiments, ONNX deployment, evaluation, and lessons learned.

Author: Ayan Humayoun

Institution: University of Karachi

Repository: GitHub repository

Date: 25 October 2026

Status: Personal technical learning report

This report documents a practical learning project. It is not peer reviewed, has not been submitted to or published by IEEE, and makes no claim of formal research acceptance.

Contents

1 Project Overview **1**

1.1 What I Wanted to Build . . . 1

1.2 Project Goals . . . 1

1.3 What I Achieved . . . 1

1.4 Tools and Environment . . . 2

2 Understanding the Game and Its Requirements **2**

2.1 How the Environment Works . . . 2

2.2 Observation and Action Representation . . . 2

2.3 Model Tensor Contract . . . 3

2.4 How Monte Carlo Tree Search Uses the Model . . . 3

2.5 What the Submission Validator Checks . . . 3

3 Designing the Neural Network **4**

3.1 Model Architecture . . . 4

3.2 Layers and Tensor Shapes . . . 4

3.3 Design Decisions and Trade-offs . . . 5

4 Generating Data and Training the Model **5**

4.1 Dataset Overview . . . 5

4.2 Random-Evaluator Self-Play . . . 6

4.3 Learning from GreedyAgent . . . 6

4.4 Guided Self-Play . . . 6

4.5 Training Objectives . . . 7

4.6 Training Configuration . . . 7

4.7 What I Learned About Data Quality . . . 8

5 Exporting and Validating the Model **8**

5.1 Exporting to ONNX . . . 8

5.2 Validator and Runtime Results . . . 9

5.3 What Validation Actually Proved . . . 9

6 Evaluating the Models **9**

6.1 Evaluation Setup . . . 9
6.2 Results . . . 10
6.3 Interpreting the Results . . . 10
6.4 What I Could Not Verify . . . 10

7 Challenges and Lessons Learned **11**
7.1 The Development Journey . . . 11
7.2 Lessons About Machine Learning . . . 11
7.3 Lessons About Engineering and Reproducibility . . . 12

8 Limitations **12**
8.1 Small Datasets and Evaluation . . . 12
8.2 Data and Training Limitations . . . 12
8.3 Unverified Reproducibility Details . . . 12
8.4 Single Hardware Setup . . . 13
8.5 Upstream Code and Licensing . . . 13

9 What I Would Do Next **13**

10 Conclusion **13**

A Commands and Reproduction Notes **14**

B Project Files and Artifacts **14**

1 Project Overview

1.1 What I Wanted to Build

This project explored whether a compact convolutional neural network could serve as the policy–value model for Colony: One, a two-team strategy game played on an 8×8 board. The tournament environment uses a referee-controlled Monte Carlo tree search (MCTS) to select actions based on a submitted model. Rather than writing a complete move-selection program, the participant supplies a neural network that evaluates positions and provides action-prior logits.

The practical goal was to understand the full workflow: reading the environment contract, implementing a model, generating training data, training the network, exporting it to the Open Neural Network Exchange (ONNX) format, validating the export, and testing the result against the environment's GreedyAgent baseline.

The work began as a contest-oriented experiment associated with the 2026 Cloudflight Coding Contest. After the contest submission period closed on 23 October 2026, the focus shifted from contest optimization to documenting the implementation, experiments, limitations, and lessons learned.

1.2 Project Goals

The goals were to:

- implement a compact CNN with a policy head and a value head;
- satisfy the environment's required input and output tensor shapes;
- experiment with random-evaluator self-play and GreedyAgent-generated data;
- export the trained model to ONNX with a dynamic batch dimension;
- run the supplied submission validator and a small baseline evaluation;
- record what worked, what did not demonstrate improvement, and what should change in a future iteration.

1.3 What I Achieved

The original model, `model.onnx`, was accepted by the local submission validator. This established that the model passed the validator's structural, runtime, and local match checks. It did not establish that the model played well.

In a preliminary comparison of 20 games against GreedyAgent using 60 MCTS simulations per move, the original model won 7 games. A model trained on GreedyAgent-generated data won 6 games in its corresponding 20-game evaluation. The recorded difference is small and does not demonstrate that either model is stronger.

The project produced model code, data-generation and training scripts, ONNX exports, small datasets, evaluation tooling, and this technical report. Some details needed for a fully reproducible

experiment—including complete raw game logs, package versions, and verified environment seeding—were not recorded.

1.4 Tools and Environment

Development was reported on an Ubuntu laptop, a Dell Latitude E7470 with a sixth-generation Intel Core i5 processor, 8 GB RAM, and a 128 GB SSD. PyTorch was used for the neural network and training, and ONNX Runtime was used by the environment to execute exported models. Exact installed Python and package versions were not recorded in the available experiment notes.

2 Understanding the Game and Its Requirements

2.1 How the Environment Works

Colony: One is a two-team strategy game played on an 8×8 board. The environment is distributed as part of the 2026 Cloudflight Coding Contest material. The upstream package provides the rules, observation encoding, environment, MCTS implementation, baseline agents, ONNX runner, and submission validator.

The model does not directly choose the final move. It evaluates an observation and returns policy logits and a scalar value. MCTS uses those outputs while searching possible continuations and selects the action according to the referee's search procedure.

2.2 Observation and Action Representation

The model input is a canonical observation tensor with shape $(B, 16, 8, 8)$, where B is the batch size. The observation is oriented to the team whose turn it is; for Magenta, the board is rotated by 180 degrees.

The 16 planes are documented as follows:

- planes 0–5: the current team's pieces;
- planes 6–11: the opponent's pieces;
- plane 12: firewall-locked squares;
- plane 13: the value `actions_left / 2`;
- plane 14: a firewall-urgency value;
- plane 15: an indicator that Magenta is to move.

An action index is computed as

$$\text{action index} = \text{from square} \times 64 + \text{to square},$$

where a square is represented as

$$\text{square} = \text{row} \times 8 + \text{column}.$$

The action space therefore contains 4,096 possible indices, although only approximately 12–30 actions are legal in a typical position according to the upstream documentation.

The environment's `step()` method applies one action rather than a full turn. Each team takes two consecutive actions before play passes to the other team. The game has no draws according to the upstream documentation, and the value target is $+1$ for a win and -1 for a loss from the side-to-move perspective.

2.3 Model Tensor Contract

The required tensor contract is summarized in Table 1. All tensors use 32-bit floating-point values, and the batch dimension must be dynamic.

Table 1: Model input and output contract.

Tensor	Direction	Shape	Type	Meaning
`observation`	Input	$(B, 16, 8, 8)$	float32	Canonical encoded board state
`policy`	Output	$(B, 4096)$	float32	Raw action logits
`value`	Output	$(B,)$	float32	Value estimate in $[-1, 1]$

2.4 How Monte Carlo Tree Search Uses the Model

MCTS searches a tree of possible game continuations. The model's policy logits provide prior information about candidate actions, while the value output estimates the position's outcome from the current team's perspective. These outputs guide the search; the model itself does not implement the game rules or the full search algorithm.

The referee uses 60 simulations per action in Level 3 and 100 in Level 4, while the local default is 300 according to the upstream documentation. Other documented settings include $c_{\text{puct}} = 1.5$, search batch size 8, virtual loss 1.0, root Dirichlet noise with $\alpha = 0.3$ and $\epsilon = 0.15$, and a changing root-action temperature during the opening actions. The referee masks illegal actions and normalizes policy logits over legal actions only. The upstream documentation also advises masking illegal actions during training.

2.5 What the Submission Validator Checks

The upstream submission contract specifies a single self-contained file of at most 5 MiB, the `ai.onnx` opset range 13–18, an allow-listed operator set, and dynamic batch axes. The local validator reports `ACCEPTED` when the file passes structural and isolated CPU-inference checks and a Level 3 match fits the published local limits.

The documented local guard values are 1.8 seconds of CPU time for the per-upload check and 13.5 seconds for a match, with resident-memory guards of 320 MiB and 425 MiB respectively.

These are environment requirements, not measures of strategic strength.

3 Designing the Neural Network

3.1 Model Architecture

The model, `TinyColonyNet`, is implemented in `tiny_colony_net.py`. It uses a shared two-layer convolutional stem and two output heads: a policy head and a value head. Let $s \in \mathbb{R}^{16\times8\times8}$ be an observation, ϕ the ReLU activation, and $\mathrm{Conv}_{3\times3}$ a 3×3 convolution.

The shared stem is represented as

$$h = \phi\left(\mathrm{Conv}^{(2)}_{3\times3}\left(\phi\left(\mathrm{Conv}^{(1)}_{3\times3}(s)\right)\right)\right). \tag{1}$$

The policy head maps the spatial features to 4,096 action logits:

$$z_p = W_p\,\mathrm{vec}(h_p) + b_p, \qquad W_p \in \mathbb{R}^{4096\times64}. \tag{2}$$

The value head pools the shared features spatially and predicts a scalar:

$$\hat{v} = \tanh\left(w_2^\top \phi\left(W_1\,\mathrm{GAP}(h) + b_1\right) + b_2\right), \tag{3}$$

where GAP denotes global average pooling. The model returns raw policy logits; softmax is applied in the training loss and by the referee as part of action selection.

3.2 Layers and Tensor Shapes

Table 2 summarizes the documented architecture. Parameter counts were derived manually from the layer definitions and were not confirmed by running a parameter-count script at the time of writing.

Table 2: Model layers and output shapes.

Block	Layer	Output shape	Parameters
Stem	Conv2d(16→16, 3×3, padding 1) + ReLU	$(B, 16, 8, 8)$	2,320
Stem	Conv2d(16→16, 3×3, padding 1) + ReLU	$(B, 16, 8, 8)$	2,320
Policy	Conv2d(16→1, 1×1) + ReLU	$(B, 1, 8, 8)$	17
Policy	Flatten	$(B, 64)$	–
Policy	Linear(64→4096)	$(B, 4096)$	266,240
Value	Adaptive average pool + flatten	$(B, 16)$	–
Value	Linear(16→16) + ReLU	$(B, 16)$	272
Value	Linear(16→1) + Tanh and squeeze	$(B,)$	17
Total			**271,186**

The derived parameter count corresponds to approximately 1.03 MiB of float32 weights, which is below the 5 MiB submission limit. The actual ONNX file size was not recorded in the available notes. The model's own shape-check script can be used to confirm the parameter count.

3.3 Design Decisions and Trade-offs

Several properties of the architecture are worth noting:

- Two stacked 3×3 convolutions give each stem feature a 5×5 receptive field on the board.
- Approximately 98.2% of the derived parameters are in the final policy layer, which maps 64 features to 4,096 logits.
- The policy head passes through a single-channel spatial bottleneck.
- The value head uses global average pooling, which discards explicit spatial position before the dense layers.

These design choices may limit the model's ability to represent useful game strategies, but no architecture ablation was performed. They should therefore be treated as design observations and hypotheses, not proven causes of the evaluation result.

4 Generating Data and Training the Model

4.1 Dataset Overview

Three small datasets were generated during the project. Table 3 summarizes their reported sizes and intended roles. These counts come from the available project records; the arrays were not re-counted for this report.

Table 3: Dataset inventory.

File	Positions	Target source	Role and caveat
`selfplay_data.npz`	87	MCTS targets with random evaluator; terminal values ± 1	Pipeline test; little strategic signal; excluded from Git.
`greedy_data.npz`	2,710	One-hot GreedyAgent actions; terminal values ± 1	Imitation and outcome regression; heuristic teacher.
`guided_selfplay_data.npz`	76	Model-guided MCTS; two games	Generation experiment only; no training script was found.

4.2 Random-Evaluator Self-Play

The script `train_selfplay.py` runs MCTS with a `RandomEvaluator` that returns standard-normal policy logits and zero value estimates. The recorded defaults include 20 simulations, search batch size 1, and root noise. The saved policy target is derived from the MCTS root distribution. Each stored position receives a terminal value of $+1$ if its team won and -1 otherwise.

The dataset contains 87 positions. This is consistent with the script's default of two games, but the exact invocation was not recorded. The random logits use NumPy's global generator, which the script does not seed; bit-for-bit regeneration is therefore not established.

Because the search prior is random and the evaluator's value estimate is zero, the data carry little strategic signal. Training on these positions can test whether the training pipeline runs, but a low training loss would not by itself show that the model learned a useful game strategy.

4.3 Learning from GreedyAgent

The script `train_greedy_data.py` generates games between two instances of GreedyAgent. Its defaults are 100 games and seed 42, with the agents seeded from a seed-42 generator. At each action, the script stores the observation and a one-hot policy target at the selected action index. Once a game ends, each stored position receives $+1$ if its team won and -1 otherwise. Games with no winner are skipped.

The resulting dataset contains 2,710 positions. The available records do not establish whether these came from exactly 100 completed games. Targets are recorded for both teams regardless of the outcome, so actions taken by losing sides are imitated too. This is heuristic imitation, not imitation of an optimal player.

4.4 Guided Self-Play

The script `train_guided_selfplay.py` is a data generator rather than a training script. It runs MCTS using an ONNX evaluator, defaults to `model.onnx`, and has defaults of five games, 20 simulations, search batch size 1, root noise enabled, and action sampling at temperature 1.0.

The recorded guided dataset contains 76 positions from two games. The invocation that generated it was not recorded. No `model_guided.onnx` file was observed, and no script that trains on this dataset was found. The dataset therefore shows that a data-generation path was run; it does not demonstrate learning from guided self-play.

4.5 Training Objectives

The policy head is trained to predict action targets. For a batch of size B, let $z_i \in \mathbb{R}^{4096}$ be the logits for sample i, $p_i(a)$ the target probability assigned to action a, $\hat{v}_i$ the predicted value, and $v_i \in \{-1, +1\}$ the terminal outcome target.

The policy loss is cross-entropy:

$$\mathcal{L}_{\text{policy}} = -\frac{1}{B} \sum_{i=1}^{B} \sum_{a=1}^{4096} p_i(a) \log \text{softmax}(z_i)_a. \tag{4}$$

The value loss is mean squared error:

$$\mathcal{L}_{\text{value}} = \frac{1}{B} \sum_{i=1}^{B} (\hat{v}_i - v_i)^2. \tag{5}$$

The two losses are added with equal weight:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + \mathcal{L}_{\text{value}}. \tag{6}$$

For GreedyAgent imitation, each target is one-hot at the teacher's chosen action a_i^*, so the policy loss simplifies to

$$\mathcal{L}_{\text{policy}} = -\frac{1}{B} \sum_{i=1}^{B} \log \text{softmax}(z_i)_{a_i^*}. \tag{7}$$

The training softmax runs over all 4,096 logits. The referee instead normalizes over legal actions only, and the upstream documentation advises masking illegal actions during training. The training objective therefore differs from the inference-time use of the policy.

4.6 Training Configuration

Both training scripts use the CPU, set PyTorch and NumPy seeds to 42, and create a seed-42 random split using a validation size of $\max(1, \lfloor 0.2N \rfloor)$. Optimization uses Adam.

Table 4: Reported training configuration.

Setting	Initial training	GreedyAgent training
Script	`train_model.py`	`train_greedy_model.py`
Dataset	`selfplay_data.npz`	`greedy_data.npz`
Batch size	16	32
Learning rate	0.001	0.0003
Epochs	40, fixed	Up to 60
Early stopping	None	Patience 8 on validation loss
Checkpoint saved	Final-epoch weights	Best-validation weights
Output	`tiny_colony_net.pth`	`tiny_colony_net_greedy.pth`
Train/validation split	70/17	2,168/542

Training loss, validation loss, best epoch, and final epoch metrics were not recorded in the available report notes and are not presented here.

4.7 What I Learned About Data Quality

The experiments highlighted that the number of positions is not the only important factor. The quality of the target policy matters. Random-evaluator labels offer little strategic guidance, while GreedyAgent labels are limited by the heuristic that generated them. Outcome labels assign the same final result to every position in a game, which can provide a noisy learning signal.

The validation split also matters. Randomly splitting individual positions can place nearby states from the same game in both training and validation sets. Because those states are correlated, validation loss may look better than performance on genuinely unseen games. A game-level split would provide a more useful estimate of generalization.

5 Exporting and Validating the Model

5.1 Exporting to ONNX

The export scripts load a checkpoint with `weights_only=True`, switch the network to evaluation mode, and trace it using a dummy input of shape $(2, 16, 8, 8)$. The tensors are named `observation`, `policy`, and `value`. Axis 0 is declared dynamic for all three tensors, and opset 17 is used with constant folding enabled and external data disabled.

The script `export_onnx.py` exports `tiny_colony_net.pth` to `model.onnx`. The script `export_greedy_onnx.py` exports `tiny_colony_net_greedy.pth` to `model_greedy.onnx`. The exact PyTorch version was not recorded or pinned in `requirements.txt`.

The original model's provenance should be treated cautiously. The intended script chain connects `model.onnx` to the checkpoint from the initial training script, but the presence of

a local `backups/` directory means that the exact weights used in the evaluation should be confirmed before attempting strict reproduction.

5.2 Validator and Runtime Results

Table 5 records the available measurements for the original `model.onnx`. These are from one recorded run and should not be treated as a benchmark.

Table 5: Recorded validator and runtime measurements for `model.onnx`.

Measurement	**Recorded value**
Validator verdict	`ACCEPTED`
Dry-run time	Approximately 127.2 ms
Peak resident memory	Approximately 82 MiB
CPU time	Approximately 0.94 s
Level 3 smoke-test match vs. GreedyAgent	40 actions; approximately 2.51 s CPU time

These values were below the published local guard limits. The available record does not map every measurement to a specific guard value. Whether `model_greedy.onnx` passed the validator was not recorded, so no claim is made about its validator status.

5.3 What Validation Actually Proved

Acceptance shows that the original file passed the validator's structural, resource, and local match checks under the tested conditions. It is an important integration milestone: the exported model could be loaded and used by the environment within the checked limits.

It does not show that the policy or value estimates are strategically useful. Runtime compatibility and game-playing strength are separate questions. A single local run is also insufficient for a reliable latency benchmark.

6 Evaluating the Models

6.1 Evaluation Setup

The script `evaluate_greedy.py` accepts a model path, number of games, number of MCTS simulations, and a base seed. Its defaults are 10 games, 60 simulations, and seed 42. For each game, it creates a `ColonyOneEnv()` instance and an `OnnxEvaluator`, configures MCTS, seeds the search generator with $s + i$, and seeds GreedyAgent with $s + i + 1000$. The model plays Cyan on even-indexed games and Magenta on odd-indexed games.

The reported comparison used 20 games and 60 simulations per move. The exact command line was not recorded; the command in the appendix is a presumed invocation based on the script's arguments.

6.2 Results

Table 6 summarizes the recorded win counts. The primary reported numbers are the raw counts, not a claim of statistical significance.

Table 6: Twenty-game comparison against GreedyAgent, using 60 simulations per move.

Model	Artifact	Games	Model wins	Reported win fraction
Original	`model.onnx`	20	7	35% (7/20)
GreedyAgent-trained	`model_greedy.onnx`	20	6	30% (6/20)
Difference	–	–	−1	−5 percentage points, descriptive only

The counts of GreedyAgent wins and unfinished games were not recorded in the available notes. The upstream checklist calls for beating GreedyAgent more often than not; both recorded model results were below 10 wins out of 20.

6.3 Interpreting the Results

The GreedyAgent-trained model recorded one fewer win than the original model. This does not show that the GreedyAgent-trained model is worse, and it does not show that the original model is better. With only 20 games, a difference of one win is too small to support a reliable conclusion.

For scale only, the standard error of a proportion near 0.35 based on 20 independent games is approximately

$$\sqrt{\frac{0.35(1-0.35)}{20}} \approx 0.11.$$

This is an order-of-magnitude illustration rather than a formal test. No confidence interval, p-value, or significance claim is made, and the two runs are not guaranteed to be independent.

6.4 What I Could Not Verify

The following details were not available in the records used to prepare this report:

- raw per-game outcomes and the counts of GreedyAgent wins or unfinished games;
- the exact command line used for the 20-game comparison;
- whether the environment constructor's default seeding is deterministic;
- the exact installed PyTorch and Python versions;
- whether the weights in the evaluated `model.onnx` exactly match the checkpoint expected from the initial training script;
- training and validation loss curves, best epochs, and complete logs.

These gaps limit strict reproducibility and are kept explicit rather than filled with assumptions.

7 Challenges and Lessons Learned

7.1 The Development Journey

The work proceeded through several practical stages:

1. Read the upstream interface documentation and established the tensor contract and one-action-per-step semantics.
2. Implemented `TinyColonyNet`, exported the original model to ONNX, and obtained validator acceptance and a Level 3 smoke match.
3. Generated 87 positions using a random evaluator and ran the initial training pipeline as an end-to-end test.
4. Recognized that random-evaluator data offered little useful strategic signal.
5. Built a GreedyAgent data generator, trained a second checkpoint with early stopping, and exported `model_greedy.onnx`.
6. Compared the two models over 20 games, recording 7 wins for the original model and 6 for the GreedyAgent-trained model.
7. Generated a small guided self-play dataset of 76 positions from two games, but did not establish a training result from it.
8. Shifted the focus after the contest deadline toward documentation, attribution, Git hygiene, and recording limitations.

7.2 Lessons About Machine Learning

The project reinforced several practical lessons:

- A model that runs correctly is not necessarily a model that plays well.
- Supervised learning cannot recover a strong strategy from uninformative labels; the quality of the teacher and target matters.
- A few dozen positions are useful for testing code paths, not for supporting broad claims about learning.
- Lower validation loss does not guarantee stronger game outcomes.
- A game-level validation split is preferable when positions from the same game are strongly correlated.
- Training with a softmax over all actions differs from the referee's legal-action normalization.
- A small match sample can be useful as a smoke test, but not as a reliable benchmark of relative strength.

7.3 Lessons About Engineering and Reproducibility

The project also showed the importance of recording exact commands, package versions, configuration, random seeds, and raw evaluation logs. Seeding some components does not prove that the complete environment is deterministic. Model filenames and checkpoints should have a clear provenance, and every reported measurement should be linked to the run that produced it.

The local virtual environment occupied approximately 5.5 GB and was excluded from Git. Caches, environment files, logs, backups, the initial self-play datasets, and checkpoint files were also excluded according to the project's Git hygiene notes. A filename scan for obvious environment, credential, and secret-named files found no matches, but that scan does not guarantee that the repository contains no sensitive information.

At one point, `python -version` returned a command-not-found message and suggested `python3` or installing `python-is-python3`. Later terminal output showed an active virtual environment. The exact resolution sequence was not recorded.

8 Limitations

8.1 Small Datasets and Evaluation

The datasets contained 87, 2,710, and 76 positions. These are small samples for a model whose policy head predicts 4,096 action logits. The evaluation used 20 games per model, and raw per-game logs were not retained in the report records. The results cannot establish a small performance difference reliably.

8.2 Data and Training Limitations

The random-evaluator dataset has weak strategic information. GreedyAgent is a heuristic teacher, and its actions are not guaranteed to be optimal. Both winning and losing sides' actions are imitated. The guided self-play dataset is too small to support a learning conclusion, and no training script using that dataset was found.

Random position-level validation splits can leak correlated positions from the same game across training and validation. The policy loss also normalizes over all 4,096 action logits rather than masking illegal actions as the referee does.

8.3 Unverified Reproducibility Details

Environment seeding, package versions, complete training logs, full evaluation outcomes, and the exact evaluated checkpoint provenance were not fully verified. The measurements should be understood as recorded results from a small development experiment, not a fully reproducible benchmark.

8.4 Single Hardware Setup

All reported local measurements came from one laptop. No comparison across machines or repeated performance benchmark was conducted.

8.5 Upstream Code and Licensing

This project uses the upstream Colony: One environment. The game engine, rules, MCTS implementation, baseline agent, ONNX runner, and validator are upstream components rather than original work by the report author. The repository's `LICENSE` names the upstream copyright holder and includes conditions relating to non-commercial use, retention of notices, commercial use, events, workshops, courses, competitions, and derivative works.

This report does not interpret those terms. The existing license and attribution text should remain unchanged. Permission and license requirements should be clarified with the copyright holder before further public redistribution where necessary.

9 What I Would Do Next

The following items are future proposals, not completed work:

1. Use game-level training and validation splits so whole games are held out.
2. Mask illegal actions during policy training to match the referee's legal-action normalization.
3. Run a larger evaluation over hundreds of games, balance the playing sides, record per-game outcomes, and compute an appropriate interval.
4. Record environment seeds explicitly and verify how randomness flows through the game, MCTS, and baseline agent.
5. Add stronger baselines, including an untrained network, the shipped random model if available, GreedyAgent, and direct model-versus-model games.
6. Generate substantially more guided self-play data and use states visited by the learner's own search.
7. Test architecture variations, such as a wider policy head or a value head that preserves more spatial information.
8. Record software versions and hardware, pin dependencies including PyTorch, and save configurations, logs, and checksums.
9. Run the submission validator on `model_greedy.onnx` and record the result.
10. Confirm relevant permission and licensing requirements for redistribution.

10 Conclusion

This project implemented a compact CNN policy–value model, generated three small datasets, trained checkpoints, exported ONNX models, and tested the original model with the environ-

ment's submission validator. The original `model.onnx` was accepted, demonstrating compatibility with the checked interface and runtime requirements.

The GreedyAgent-trained model won 6 of 20 games in the recorded comparison, compared with 7 of 20 for the original model. The experiment did not demonstrate an improvement from training on GreedyAgent data, and it cannot establish that either model is stronger. The guided self-play data-generation path also produced too little data to support a learning conclusion.

The main value of the project is the practical experience gained from implementing the model interface, experimenting with data generation and training, exporting and validating ONNX, running a baseline comparison, and documenting limitations honestly. The next iteration should focus on better targets, legal-action masking, game-level validation, larger controlled evaluations, and stronger reproducibility practices.

A Commands and Reproduction Notes

The commands below reflect script defaults or presumed invocations. The original invocation was not recorded for rows marked with an asterisk.

Table 7: Commands and expected outputs.

Step	**Command**	**Output**
Random-evaluator data	`python train_selfplay.py`	`selfplay_data.npz`
Initial training	`python train_model.py`	`tiny_colony_net.pth`
Export original model	`python export_onnx.py`	`model.onnx`
Validate original model	`python tools/validate_submission.py model.onnx`	Validator verdict
GreedyAgent data	`python train_greedy_data.py -games 100 -seed 42`	`greedy_data.npz`
GreedyAgent training	`python train_greedy_model.py`	`tiny_colony_net_greedy.pth`
Export GreedyAgent model	`python export_greedy_onnx.py`	`model_greedy.onnx`
Evaluation*	`python evaluate_greedy.py -model model.onnx -games 20 -simulations 60 -seed 42`	Outcome counts
Guided data*	`python train_guided_selfplay.py -model model.onnx -games N -simulations S -seed 42`	`guided_selfplay_data.npz`

B Project Files and Artifacts

Table 8 summarizes the reported repository layout as of 25 October 2026. The artifact inventory should be checked against the current repository before using it as a definitive file manifest.

Table 8: Project artifact map.

Artifact	Role	In Git?	Origin
`colony_one/`, `docs/`, `play/`, `tools/`, `LICENSE`	Environment, docs, and tools	Yes	Upstream
`tiny_colony_net.py`, `train_*.py`, `export_*.py`, `evaluate_greedy.py`	Experimental code	Yes	Author's experiment
`greedy_data.npz`, `guided_selfplay_data.npz`	Datasets	Yes	Generated during project
`model.onnx`, `model_greedy.onnx`	Exported models	Yes	Generated during project
`selfplay_data*.npz`, `*.pth`, `backups/`	Local artifacts	No, reported ignored	Local experiment

References and Further Reading

The following sources provide context for the environment and the technical methods mentioned in this report. The list is retained from the project notes; bibliographic details and current URLs should be checked before formal reuse.

[1] Cloudflight GmbH, "The Cloudflight Coding Contest." Available: `https://www.cloudflight.io/en/the-cloudflight-coding-contest/` (accessed 25 October 2026).

[2] P. Puntschart, "Colony: One tournament environment," including the repository README, `docs/observation.md`, `docs/submission.md`, and `LICENSE`, as included in the repository snapshot. Available: `https://github.com/ayan-humayoun56/colony-one-neural-network` (accessed 25 October 2026).

[3] R. Coulom, "Efficient selectivity and backup operators in Monte-Carlo tree search," in *Computers and Games (CG 2006)*, Lecture Notes in Computer Science. Springer, 2007, pp. 72–83. Available: `https://remi-coulom.fr/CG2006`.

[4] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Machine Learning: ECML 2006*, Lecture Notes in Computer Science, vol. 4212. Springer, 2006, pp. 282–293. doi: 10.1007/11871842_29.

[5] C. B. Browne et al., "A survey of Monte Carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, March 2012. doi: 10.1109/TCIAIG.2012.2186810.

[6] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, January 2016. doi: 10.1038/nature16961.

[7] D. Silver et al., "Mastering the game of Go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, October 2017. doi: 10.1038/nature24270.

[8] D. Silver et al., "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, December 2018. doi: 10.1126/science.aar6404.

[9] ONNX, "Open Neural Network Exchange." Available: `https://onnx.ai/` (accessed 25 October 2026).

[10] ONNX Runtime, "Welcome to ONNX Runtime." Available: `https://onnxruntime.ai/docs/` (accessed 25 October 2026).

[11] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. arXiv:1912.01703.

[12] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, PMLR, vol. 15, 2011, pp. 627–635. Available: `https://proceedings.mlr.press/v15/ross11a`.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[14] PyTorch contributors, "torch.onnx," PyTorch documentation. Available: `https://docs.pytorch.org/docs/stable/onnx.html` (accessed 25 October 2026).